

Web System Technologies

IT0049



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 4

View

View

A **view** is simply a web page, or a page fragment, like a header, footer, sidebar, etc. In fact, views can flexibly be embedded within other views if you need this type of hierarchy.

Views are never called directly, they must be loaded by a controller or [view route](#).

Remember that in an MVC framework, the Controller acts as the traffic cop, so it is responsible for fetching a particular view. If you have not read the [Controllers](#) page, you should do so before continuing.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Creating a View

Using the example controller you created in the controller page, let's add a view to it.

Using your text editor, create a file called **blog_view.php** and put this in it

```
<html>
  <head>
    <title>My Blog</title>
  </head>
  <body>
    <h1>Welcome to my Blog!</h1>
  </body>
</html>
```

Then save the file in your app/Views directory.

Displaying a View

To load and display a particular view file you will use the [view\(\)](#) function like following code in your controller:

```
return view('name');
```

Where *name* is the name of your view file.

Note: If the file extension is omitted, then the views are expected to end with the **.php** extension.

Displaying a View

Now, create a file called **Blog.php** in the **app/Controllers** directory, and put this in it:

```
<?php

namespace App\Controllers;

class Blog extends BaseController
{
    public function index()
    {
        return view('blog_view');
    }
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Displaying a View

Open the routing file located at **app/Config/Routes.php**, and look for the “Route Definitions”. Add the following code:

```
use App\Controllers\Blog;  
$routes->get('blog', [Blog::class, 'index']);
```

If you visit your site, you should see your new view. The URL was similar to this:

example.com/index.php/blog/

Loading Multiple Views

CodeIgniter will intelligently handle multiple calls to [view\(\)](#) from within a controller. If more than one call happens they will be appended together.

For example, you may wish to have a header view, a menu view, a content view, and a footer view. That might look something like this:

In the example, we are using “dynamically added data”.

```
<?php

namespace App\Controllers;

use CodeIgniter\Controller;

class Page extends Controller
{
    public function index()
    {
        $data = [
            'page_title' => 'Your title',
        ];

        return view('header')
            . view('menu')
            . view('content', $data)
            . view('footer');
    }
}
```

Storing Views within Sub-directories

Your view files can also be stored within sub-directories if you prefer that type of organization. When doing so you will need to include the directory name loading the view. Example:

```
return view('directory_name/file_name');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Adding Dynamic Data to the View

Data is passed from the controller to the view by way of an array in the second parameter of the [view\(\)](#) function.

Here's an example:

```
$data = [  
    'title'    => 'My title',  
    'heading' => 'My Heading',  
    'message' => 'My Message',  
];  
  
return view('blog_view', $data);
```

Let's try it with your controller file. Open it and add this code:

```
<?php  
  
namespace App\Controllers;  
  
class Blog extends BaseController  
{  
    public function index()  
    {  
        $data['title']    = 'My Real Title';  
        $data['heading'] = 'My Real Heading';  
  
        return view('blog_view', $data);  
    }  
}
```



Adding Dynamic Data to the View

Now open your view file and change the text to variables that correspond to the array keys in your data:

```
<html>
  <head>
    <title><?= esc($title) ?></title>
  </head>
  <body>
    <h1><?= esc($heading) ?></h1>
  </body>
</html>
```

Then load the page at the URL you've been using and you should see the variables replaced.

Creating Loops

You can pass multi dimensional arrays, which can be looped to generate multiple rows. Here's a simple example. Add this to your controller:

```
<?php

namespace App\Controllers;

class Blog extends BaseController
{
    public function index()
    {
        $data = [
            'todo_list' => ['Clean House', 'Call Mom', 'Run Errands'],
            'title'      => 'My Real Title',
            'heading'    => 'My Real Heading',
        ];

        return view('blog_view', $data);
    }
}
```

Now open your view file and create a loop:

```
<html>
<head>
    <title><?= esc($title) ?></title>
</head>
<body>
    <h1><?= esc($heading) ?></h1>

    <h2>My Todo List</h2>

    <ul>
        <?php foreach ($todo_list as $item): ?>

            <li><?= esc($item) ?></li>

        <?php endforeach ?>
    </ul>

</body>
</html>
```

Module 4

Model

Model

The CodeIgniter's Model provides convenience features and additional functionality that people commonly use to make working with a **single table** in your database more convenient.

It comes out of the box with helper methods for much of the standard ways you would need to interact with a database table, including finding records, updating records, deleting records, and more.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Accessing Models

Models are typically stored in the **app/Models** directory. They should have a namespace that matches their location within the directory, like namespace `App\Models`.

You can access models within your classes by creating a new instance or using the [`model\(\)`](#) helper function.

```
<?php

// Create a new class manually.
$userModel = new \App\Models\UserModel();

// Create a shared instance of the model.
$userModel = model('UserModel');
// or
$userModel = model('App\Models\UserModel');
// or
$userModel = model(\App\Models\UserModel::class);

// Create a new class with the model() function.
$userModel = model('UserModel', false);

// Create shared instance with a supplied database connection.
$db          = db_connect('custom');
$userModel = model('UserModel', true, $db);
```



Connecting to a Database

Connecting to the Default Group

You can connect to your database by adding this line of code in any function where it is needed, or in your class constructor to make the database available globally in that class.

```
$db = \Config\Database::connect();
```

If the above function does **not** contain any information in the first parameter, it will connect to the default group specified in your database config file. A convenience method exists that is purely a wrapper around the above line and is provided for your convenience:

```
$db = db_connect();
```

Connecting to a Database

Connecting to Specific Group

The first parameter of this function can **optionally** be used to specify a particular database group from your config file.

To choose a specific group from your config file you can do this:

```
$db = \Config\Database::connect('group_name');
```

Where group_name is the name of the connection group from your config file.



Connecting to a Database

Connecting to Multiple Databases

If you need to connect to more than one database simultaneously you can do so as follows:

```
$db1 = \Config\Database::connect('group_one');  
$db2 = \Config\Database::connect('group_two');
```

Note: Change the words group_one and group_two to the specific group names you are connecting to.

Usage Examples

Standard Query With Multiple Results (Object Version)

```
<?php

$query    = $db->query('SELECT name, title, email FROM my_table');
$results  = $query->getResult();

foreach ($results as $row) {
    echo $row->title;
    echo $row->name;
    echo $row->email;
}

echo 'Total Results: ' . count($results);
```

The above getResult() function returns an array of **objects**.
Example: \$row->title

Usage Examples

Standard Query With Multiple Results (Array Version)

```
<?php

$query    = $db->query('SELECT name, title, email FROM my_table');
$results = $query->getResultArray();

foreach ($results as $row) {
    echo $row['title'];
    echo $row['name'];
    echo $row['email'];
}
```

The above getResultArray() function returns an array of standard array indexes.

Example: \$row['title']

Usage Examples

Standard Query With Single Result

```
<?php  
  
$query = $db->query('SELECT name FROM my_table LIMIT 1');  
$row   = $query->getRow();  
echo $row->name;
```

The above **getRow()** function returns an **object**.
Example: **\$row->name**



Usage Examples

Standard Query With Single Result (Array version)

```
<?php  
  
$query = $db->query('SELECT name FROM my_table LIMIT 1');  
$row   = $query->getRowArray();  
echo $row['name'];
```

The above **getRowArray()** function returns an **array**.
Example: **\$row['name']**.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Usage Examples

Standard insert

```
<?php
```

```
$sql = 'INSERT INTO mytable (title, name) VALUES (' . $db->escape($title) . ', ' . $db->escape($name) . ')';  
$db->query($sql);  
echo $db->affectedRows();
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder

The [Query Builder](#) gives you a simplified means of retrieving data:

```
<?php

$query = $db->table('table_name')->get();

foreach ($query->getResult() as $row) {
    echo $row->title;
}
```

The above **get()** function retrieves all the results from the supplied table. The [Query Builder](#) class contains a full complement of functions for working with data.

Query Builder: Examples

\$builder->get()

Runs the selection query and returns the result. Can be used by itself to retrieve all records from a table:

```
<?php  
  
$builder = $db->table('mytable');  
$query   = $builder->get(); // Produces: SELECT * FROM mytable
```

The first and second parameters enable you to set a limit and offset clause:

```
<?php  
  
$query = $builder->get(10, 20);  
/*  
 * Executes: SELECT * FROM mytable LIMIT 20, 10  
 * (in MySQL. Other databases have slightly different syntax)  
 */
```

You'll notice that the method is assigned to a variable named `$query`, which can be used to show the results:

```
<?php  
  
$query = $builder->get();  
  
foreach ($query->getResult() as $row) {  
    echo $row->title;  
}
```



Query Builder: Examples

\$builder->getWhere()

Identical to the **get()** method except that it permits you to add a “where” clause in the first parameter, instead of using the **\$builder->where()** method:

```
<?php  
  
$query = $builder->getWhere(['id' => $id], $limit, $offset);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->select()

Permits you to write the **SELECT** portion of your query:

```
<?php  
  
$builder->select('title, content, date');  
$query = $builder->get();  
// Executes: SELECT title, content, date FROM mytable
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->from()

Permits you to write the **FROM** portion of your query:

```
<?php

$builder = $db->table('users');
$builder->select('title, content, date');
$builder->from('mytable');
$query = $builder->get();
// Produces: SELECT title, content, date FROM users, mytable
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->join()

Permits you to write the **JOIN** portion of your query:

```
<?php

$builder = $db->table('blogs');
$builder->select('*');
$builder->join('comments', 'comments.id = blogs.id');
$query = $builder->get();
/*
 * Produces:
 * SELECT * FROM blogs JOIN comments ON comments.id = blogs.id
 */
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->where()

This method enables you to set **WHERE** clauses using one of five methods:

1. Simple key/value method

```
<?php  
  
$builder->where('name', $name);  
// Produces: WHERE name = 'Joe'
```

Notice that the equal sign is added for you.

If you use multiple method calls they will be chained together with **AND** between them:

```
<?php  
  
$builder->where('name', $name);  
$builder->where('title', $title);  
$builder->where('status', $status);  
// WHERE name = 'Joe' AND title = 'boss' AND status = 'active'
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->where()

This method enables you to set **WHERE** clauses using one of five methods:

2. Custom key/value method

You can include an operator in the first parameter in order to control the comparison:

```
<?php

$builder->where('name !=', $name);
$builder->where('id <', $id);
// Produces: WHERE name != 'Joe' AND id < 45
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

`$builder->where()`

This method enables you to set **WHERE** clauses using one of five methods:

3. Associative array method

```
<?php  
  
$array = ['name' => $name, 'title' => $title, 'status' => $status];  
$builder->where($array);  
// Produces: WHERE name = 'Joe' AND title = 'boss' AND status = 'active'
```

You can include your own operators using this method as well:

```
<?php  
  
$array = ['name !=' => $name, 'id <' => $id, 'date >' => $date];  
$builder->where($array);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->where()

This method enables you to set **WHERE** clauses using one of five methods:

4. Custom string

You can write your own clauses manually:

```
<?php  
  
$where = "name='Joe' AND status='boss' OR status='active'";  
$builder->where($where);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->where()

This method enables you to set **WHERE** clauses using one of five methods:

5. RawSql

Since v4.2.0, \$builder->where() accepts a CodeIgniter\Database\RawSql instance, which expresses raw SQL strings.

```
<?php

use CodeIgniter\Database\RawSql;

$sql = "id > 2 AND name != 'Accountant'";
$builder->where(new RawSql($sql));
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->orWhere()

This method is identical to the one above, except that multiple instances are joined by **OR**:

```
<?php

$builder->where('name !=', $name);
$builder->orWhere('id >', $id);
// Produces: WHERE name != 'Joe' OR id > 50
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->whereIn()

Generates a **WHERE** field IN ('item', 'item') SQL query joined with **AND** if appropriate:

```
<?php  
  
$names = ['Frank', 'Todd', 'James'];  
$builder->whereIn('username', $names);  
// Produces: WHERE username IN ('Frank', 'Todd', 'James')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->orWhereIn()

Generates a **WHERE** field **IN** ('item', 'item') SQL query joined with **OR** if appropriate:

```
<?php  
  
$names = ['Frank', 'Todd', 'James'];  
$builder->orWhereIn('username', $names);  
// Produces: OR username IN ('Frank', 'Todd', 'James')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->whereNotIn()

Generates a **WHERE** field **NOT IN** ('item', 'item') SQL query joined with **AND** if appropriate:

```
<?php  
  
$names = ['Frank', 'Todd', 'James'];  
$builder->whereNotIn('username', $names);  
// Produces: WHERE username NOT IN ('Frank', 'Todd', 'James')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->orWhereNotIn()

Generates a **WHERE** field **NOT IN** ('item', 'item') SQL query joined with **OR** if appropriate:

```
<?php  
  
$names = ['Frank', 'Todd', 'James'];  
$builder->orWhereNotIn('username', $names);  
// Produces: OR username NOT IN ('Frank', 'Todd', 'James')
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->insert()

Generates an insert string based on the data you supply, and runs the query. You can either pass an **array** or an **object** to the method.

Here is an example using an array:

```
<?php
use CodeIgniter\Database\RawSql;

$data = [
    'id'           => new RawSql('DEFAULT'),
    'title'        => 'My title',
    'name'         => 'My Name',
    'date'         => '2022-01-01',
    'last_update' => new RawSql('CURRENT_TIMESTAMP()'),
];

$builder->insert($data);
/* Produces:
    INSERT INTO mytable (id, title, name, date, last_update)
    VALUES (DEFAULT, 'My title', 'My name', '2022-01-01', CURRENT_TIMESTAMP())
*/
```

Query Builder: Examples

\$builder->insertBatch()

Generates an insert string based on the data you supply, and runs the query. You can either pass an **array** or an **object** to the method. Here is an example using an array:

```
<?php

$data = [
    [
        'title' => 'My title',
        'name'   => 'My Name',
        'date'   => 'My date',
    ],
    [
        'title' => 'Another title',
        'name'   => 'Another Name',
        'date'   => 'Another date',
    ],
];

$builder->insertBatch($data);

/*
 * Produces:
 * INSERT INTO mytable (title, name, date)
 *     VALUES ('My title', 'My name', 'My date'),
 *     ('Another title', 'Another name', 'Another date')
 */
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->replace()

This method executes a **REPLACE** statement, which is basically the SQL standard for (optional) **DELETE + INSERT**, using *PRIMARY* and *UNIQUE* keys as the determining factor.

```
<?php

$data = [
    'title' => 'My title',
    'name'  => 'My Name',
    'date'  => 'My date',
];

$builder->replace($data);
// Executes: REPLACE INTO mytable (title, name, date) VALUES ('My title', 'My name', 'My date')
```

Query Builder: Examples

\$builder->update()

Generates an update string and runs the query based on the data you supply. You can pass an **array** or an **object** to the method. Here is an example using an array:

```
<?php

$data = [
    'title' => $title,
    'name'   => $name,
    'date'   => $date,
];

$builder->where('id', $id);
$builder->update($data);
/*
 * Produces:
 * UPDATE mytable
 * SET title = '{$title}', name = '{$name}', date = '{$date}'
 * WHERE id = $id
 */
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Query Builder: Examples

\$builder->updateBatch()

Generates an update string based on the data you supply, and runs the query. You can either pass an **array** or an **object** to the method. Here is an example using an array:

```
<?php
$data = [
    [
        'title' => 'Title 1',
        'author' => 'Author 1',
        'name'   => 'Name 1',
        'date'   => 'Date 1',
    ],
    [
        'title' => 'Title 2',
        'author' => 'Author 2',
        'name'   => 'Name 2',
        'date'   => 'Date 2',
    ],
];
$builder->updateBatch($data, ['title', 'author']);
/*
 * Produces:
 * UPDATE `mytable`
 * INNER JOIN (
 * SELECT 'Title 1' `title`, 'Author 1' `author`, 'Name 1' `name`, 'Date 1' `date` UNION ALL
 * SELECT 'Title 2' `title`, 'Author 2' `author`, 'Name 2' `name`, 'Date 2' `date`
 * ) `u`
 * ON `mytable`.`title` = `u`.`title` AND `mytable`.`author` = `u`.`author`
 * SET
 * `mytable`.`title` = `u`.`title`,
 * `mytable`.`name` = `u`.`name`,
 * `mytable`.`date` = `u`.`date`
 */
```



Query Builder: Examples

\$builder->delete()

Generates a **DELETE** SQL string and runs the query.

```
<?php

$builder->delete(['id' => $id]);
// Produces: DELETE FROM mytable WHERE id = $id
```

The first parameter is the where clause. You can also use the **where()** or **orWhere()** methods instead of passing the data to the first parameter of the method:

```
<?php

$builder->where('id', $id);
$builder->delete();
/*
 * Produces:
 * DELETE FROM mytable
 * WHERE id = $id
 */
```

Query Builder: Examples

\$builder->deleteBatch()

Generates a batch **DELETE** statement based on a set of data.

```
<?php

$data = [
    [
        'order' => 48372,
        'line'   => 3,
        'product' => 'Keyboard',
        'qty'    => 1,
    ],
    [
        'order' => 48372,
        'line'   => 4,
        'product' => 'Mouse',
        'qty'    => 1,
    ],
    [
        'order' => 48372,
        'line'   => 5,
        'product' => 'Monitor',
        'qty'    => 2,
    ],
];

$builder->setData($data, true, 'del')
->onConstraint('order, line')
->where('del.qty >', 1)
->deleteBatch();

/*
 * MySQL Produces:
 * DELETE `order_line` FROM `order_line`
 * INNER JOIN (
 * SELECT 3 `line`, 48372 `order`, 'Keyboard' `product`, 1 `qty` UNION ALL
 * SELECT 4 `line`, 48372 `order`, 'Mouse' `product`, 1 `qty` UNION ALL
 * SELECT 5 `line`, 48372 `order`, 'Monitor' `product`, 2 `qty`
 * ) `del`
 * ON `order_line`.`order` = `del`.`order` AND `order_line`.`line` = `del`.`line`
 * WHERE `del`.`qty` > 1
 */
```



Query Builder: Examples

`$builder->emptyTable()`

Generates a **DELETE** SQL string and runs the query:

```
<?php

$builder->emptyTable('mytable');
// Produces: DELETE FROM mytable
```

`$builder->truncate()`

Generates a **TRUNCATE** SQL string and runs the query.

```
<?php

$builder->truncate();
/*
 * Produce:
 * TRUNCATE mytable
 */
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

